

Your desk-side Claude Code guide

01 / START HERE VS Code extension first. No coding course required.

Keep this PDF beside VS Code. Copy a shaded request into Claude's message box, change the details, and send it. These are ordinary words, not programming. Try one task at a time.

Three places, three different jobs

Explorer	VS Code's file list. Your project is simply a folder of related work.
Claude message box	Ask questions, attach file references, and request saved outputs.
Command Palette	A search box for VS Code actions. Press Ctrl+Shift+P (Mac: Cmd+Shift+P). Do not paste work prompts here.

First session: open, connect, point to your files

1. Open VS Code. Choose **File > Open Folder** and select your practice folder. Use your company-approved storage for real work.
2. Open Extensions: **Ctrl+Shift+X** (Mac: **Cmd+Shift+X**). Find **Claude Code** by **Anthropic**, then Install. Follow your employer's sign-in/setup instructions.
3. In the Command Palette, search **Claude Code: Open in New Tab**. Complete Sign in if shown. The chat panel is your main workspace.
4. Put practice files in this folder using Windows File Explorer or Mac Finder. In Claude, type @ and select a file or folder suggestion. A file reference tells Claude where to look.

First request - paste into Claude

I am new to Claude Code. Look at this project folder. Explain what is here in plain English and suggest one small practice task. Do not change any files yet.

Keyboard basics

Copy / paste / save	Windows: Ctrl+C / Ctrl+V / Ctrl+S Mac: Cmd+C / Cmd+V / Cmd+S
Send / new line	Enter sends by default. Shift+Enter adds a line. Click inside Claude's message box first.

Reference: [VS Code interface](#) | [Official extension guide](#)

Add skills once. Then just ask.

02 / SKILLS A reusable procedure Claude can choose for your task.

A **skill** is a set of instructions and sometimes supporting files. A **plugin** bundles capabilities. A **marketplace** lists plugins you can install. Adding the list does not install its plugins.

Install through the extension GUI

1. In Claude, click / > **Customize > Plugins** (or Manage plugins). You can also type **/plugins** to open the dialog.
2. Select **Marketplaces**. Paste one source below into the repository field and click **Add**. Repeat for the second source.

Office source	anthropics/skills
Course source	jasonjeske/vscode-claude-code

3. Select **Plugins**. Find and Install **document-skills** from Anthropic and **property-tax-workbench** from property-tax-learning. Choose **Install locally** for this practice repository, or **Install for you** for all your projects if allowed.
4. Follow the restart banner if shown. Confirm both plugins are enabled. Use page 6 if the panel needs refreshing.

Your small starter toolkit

Office	xlsx : spreadsheet work. docx : Word documents. pptx : slides. pdf : PDF reading and creation.
Property-tax workbench	Workbook review, reconciliation checks, tax research, financial dashboards, prompt coaching, and structured work requests.
Optional course helpers	Writing together, creating skills, guided learning, organizing files, and research writing. Add these when useful; the course lists the selected sources.

Check readiness

Which installed skills can help with Excel, Word, PowerPoint, PDFs, and reconciliation? Separate what is available from what needs setup. Suggest one tiny test using invented data.

Use normal language. Claude can select relevant skills from their descriptions; automatic selection is not guaranteed. If needed, ask it to check its available skills. No long skill command is required.

Getting more: use the [course's five-helper checklist](#). Keep complete skill folders and licenses. Skills do not supply Office licenses, browser control, or access to company systems.

Reference: [Plugin installation](#) | [Anthropic skills](#) | [Skill behavior](#)

Talk to Claude like a colleague

03 / PROMPTS Outcome + files + rules + deliverable + checks.

Be specific about the result, not the technology. Say who will read it, which files to use, what must stay unchanged, and how you will know it is right. Replace words in [brackets] before sending.

The reusable work request

Help me [outcome] for [audience]. Use [file or folder]. Keep the originals unchanged. Save [deliverable] in outputs. Check [totals, counts, or other success measure]. First explain your approach and ask only questions that would change the result.

When you do not know what to ask

Help me turn this into a clear request: [describe the task in your own words]. Ask me up to three essential questions, one at a time. Then give me a prompt I can review and use.

Useful follow-ups - say exactly what you need

Learn	Explain that as if this is my first day. Give me one example and one small exercise.
Plan	Break this into three steps. Tell me what each step will produce.
Proceed	Proceed with that plan. Preserve the source files and save new outputs.
Correct	Use tax year 2026, not 2025. Recheck every output affected by that change.
Verify	Show the source file, sheet, and row behind each total. Flag anything you could not verify.
Simplify	Make this shorter and easier to read. Keep the numbers and qualifications.
Improve	What is the weakest part of this result? Fix it and explain the change.
Finish	List the files you saved, the checks that passed, and what still needs my review.

Save a longer request instead of repeatedly pasting it

Save our agreed task, source files, output names, and success checks in TASK.md. Use plain English. Next time I want to ask you to read TASK.md and continue.

.md means Markdown: an ordinary text document you can edit in VS Code. Chat history is useful, but a saved task file gives the next session a clear starting point.

From spreadsheets to a meeting

04 / OFFICE WORK Request it. Open it. Check it. Improve it.

Use a practice folder with **inputs** for source copies and **outputs** for results. Tell Claude your actual folder name if it differs. Run these requests separately; review each result before the next.

1 / Combine Excel files and reconcile

Inspect the Excel files in inputs. Identify sheets, columns, tax years, and matching keys before combining. Keep parcel IDs as text and originals unchanged. Save outputs/reconciliation.xlsx with source references, duplicates, unmatched items, differences, and a Checks sheet. Reconcile counts and totals back to every input. Separate net difference from total absolute differences.

Your check: open the workbook in Excel. Read Checks, inspect exceptions, and trace one record to each input. Ask Claude to explain failed checks before moving on.

2 / Turn checked results into a dashboard

Use outputs/reconciliation.xlsx to create outputs/dashboard.html for our review meeting. Show book total, bill total, net difference, exception count, and reconciliation status. Add a state comparison chart and useful filters. Explain metric definitions and check every number and filter total against the workbook. Make it readable on a meeting screen and label it with the data date.

Your check: open the HTML file in a browser. Compare headline totals and one filtered view with Excel. The report is a snapshot; ask Claude to rebuild it when inputs change.

3 / Prepare the meeting pack

Use the checked workbook to save outputs/briefing.docx and outputs/meeting.pptx. Make a one-page briefing and five slides: summary, state comparison, exceptions, decisions needed, next actions. Use consistent numbers, readable charts, and clear source notes. Separate verified findings from unresolved items.

Your check: open the files in Word and PowerPoint. Check page breaks, chart labels, readability, and agreement with Excel. VS Code is for project files and instructions; use Office to review Office layouts.

4 / Extract a PDF into Excel

Read the PDF I reference. Extract its property identifiers and amounts into outputs/pdf-extract.xlsx. Include the PDF page for every row. Flag unclear text or scanned pages; do not guess. Reconcile extracted amounts to printed totals where available.

Your check: compare a few extracted rows and the totals against the original PDF. If tools or dependencies are missing, ask Claude what is needed and use your approved setup process.

Research, remember, reuse

05 / BUILD GOOD HABITS Save evidence and successful procedures.

Research property tax with a useful saved result

Research [question] for [state and county], [property type], and [tax year]. Use current official government sources. Separate enacted rules from proposals and explain applicability. Save outputs/research.md with findings, exact source links, supporting sections, dates checked, and unresolved questions. If browsing is unavailable, tell me; do not invent current rules.

Bring reviewed research into your work

Use outputs/research.md to add a Research sheet to a new copy of our workbook. Include finding, jurisdiction/year, source link, date checked, and review status. Add a short research note to the dashboard. Keep financial totals unchanged unless I approve a specific, supported adjustment.

Before relying on a deadline or rule: open its official source and confirm the jurisdiction, year, property type, and applicable exceptions. Mark unresolved interpretations for professional review.

Turn a successful routine into a skill

Project skills live in `.claude/skills/name/SKILL.md`. Personal skills normally live under `~/ .claude/skills`; `~` means your home folder. Claude can create the folder and file for you.

Create a small skill - no coding required

Save our successful procedure as `.claude/skills/monthly-review/SKILL.md`. Include a YAML header with name and description explaining when to use it automatically, then a short checklist, output names, and checks. Exclude client data. Test on invented data and show the result.

Improve an existing skill

Add a duplicate parcel-ID check to our monthly-review skill, scoped by jurisdiction and tax year. Back up the skill and show the change. Test one duplicate and one valid repeat across years. Preserve the upstream plugin.

Keep supporting files with the skill. Modern Claude Code detects many edits automatically. Restart if a new skills directory is not recognized; then try a normal work request.

Keep preferences separate from procedures

CLAUDE.md holds project instructions; `~/ .claude/CLAUDE.md` holds personal instructions across projects. Ask Claude to merge your preferences, preserve existing instructions, and show the file for review. Do not paste prose into settings.json. Keep detailed procedures in skills.

Reference: [Create and update skills](#) | [CLAUDE.md and memory](#)

Commands, recovery, and tomorrow

06 / QUICK REFERENCE Use the GUI first; short commands are optional.

Type **/** in Claude to browse commands, or use **Customize > Slash commands** in newer versions. A terminal icon opens that action in the terminal. Availability varies by version and company setup.

/plugins	Open the extension's plugin manager.
/compact	Summarize a long conversation to free context while continuing the task. Save important decisions first.
/btw	Ask a side question. Example: <code>/btw What is a reconciliation?</code>
/usage	Account and usage details when signed in through Claude.ai. Not offered on third-party provider connections.
More, if listed	/help: help. /skills: available skills. /context: context use. /clear: fresh conversation, not a file rollback. Use GUI controls if a command is absent.

Daily GUI controls

Plan first: use the mode selector below the message box and choose Plan. Choose Manual when you want approval prompts while working. **New task:** Command Palette > Claude Code: Open in New Tab. **Resume:** Session history. **Model:** click the model name or use Switch model in the command menu.

Stop, close, refresh: they are different

Stop: click Stop and wait. Existing file changes remain. **Close:** after work stops and files are saved, click the tab's X. Closing a tab is not a reliable way to cancel work.

Refresh a stuck panel or missing installation:

1. Save edited files with **Ctrl+S** (Mac: **Cmd+S**).
2. Open the Command Palette: **Ctrl+Shift+P** (Mac: **Cmd+Shift+P**).
3. Search **Developer: Reload Window** and press Enter.
4. Reopen Claude Code. Resume through Session history, or open a new conversation to check the installed skills.

Still stuck? Quit and reopen VS Code: Windows **File > Exit**; Mac **Code > Quit Visual Studio Code**. Reopen your project. For sign-in or company connection errors, send IT the error text with credentials and private content removed.

Before you finish for the day

Save NEXT-STEPS.md with our goal, input and output paths, verified checks, unresolved items, and the next action. Tomorrow I will ask you to read it and continue.

Optional terminal: Command Palette > **Claude Code: Open in Terminal**. Once Claude starts, use the same ordinary prompts. **Ctrl+C** interrupts; **Ctrl+D** exits. Using Claude in a plain shell requires the standalone CLI.

Reference: [Extension controls](#) | [Command reference](#) | [Terminal keys](#)